

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

		Duration : 45
Minutes		
QUESTIONS: 2		Total
Marks:20		

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I _____BAIRAGANI DINESH-192372189_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:BAIRAGANI DINESH

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts

Experiment al Design	30	Well-planned, systematic implementation with optimal solution	Correct implement ation with minor issues	Basic implement ation with errors or inefficienci es	Incorrect or incomplete implement ation
Use of Tools	20	Effective and advanced use of tools/technologi es with proper configuration	Appropriate use of tools with correct execution	Limited or inconsisten t use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretati on	Basic analysis with limited insights	Poor or incorrect analysis of results
Documenta tion	10	Clear, well- structured documentation with diagrams, code, and results	Organized documenta tion with necessary details	Basic documenta tion with missing details	Poor or incomplete documenta tion

Experiment 1 — Decision Tree Classification

Dataset: Iris dataset

Criterion: Gini Index

Train/Test split: 80/20

(a) Load, preprocess and split

```
import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

# Load dataset
```

```
iris = load_iris()

X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name="target")

# Display first 5 rows
print("First 5 rows:")
print(X.head())

# Display data types
print("\nData Types:")
print(X.dtypes)

# Check missing values
print("\nMissing Values:")
print(X.isnull().sum())

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
1s print("Testing samples:", len(X_test))

... First 5 rows:
      sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0          5.1           3.5           1.4           0.2
1          4.9           3.0           1.4           0.2
2          4.7           3.2           1.3           0.2
3          4.6           3.1           1.5           0.2
4          5.0           3.6           1.4           0.2

Data Types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
dtype: object

Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
dtype: int64

Training samples: 120
Testing samples: 30
```

(b) Build Decision Tree

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
import matplotlib.pyplot as plt
```

```
# Create Decision Tree using Gini Index
```

```
model = DecisionTreeClassifier(
```

```
    criterion="gini",
```

```
    random_state=42
```

```
)
```

```
# Train
```

```
model.fit(X_train, y_train)
```

```
# Print tree structure
```

```
print("\nDecision Tree Structure:")
```

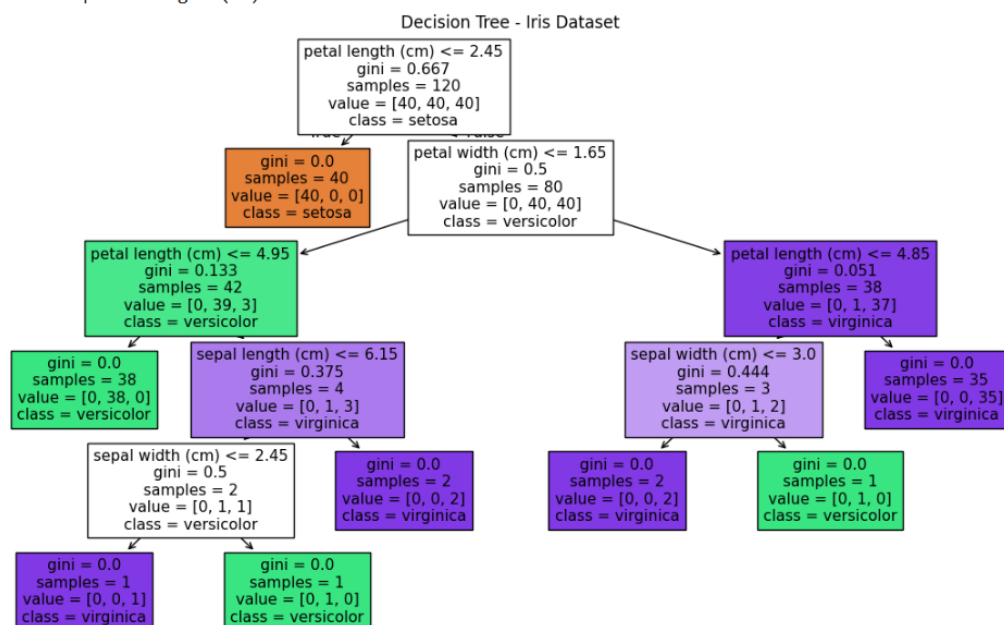
```
from sklearn.tree import export_text
```

```
print(export_text(model, feature_names=list(X.columns)))
```

```
# Root node
root_feature = X.columns[model.tree_.feature[0]]
print("\nRoot Node:", root_feature)
```

```
# Plot tree
plt.figure(figsize=(14, 8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=iris.target_names,
    filled=True
)
plt.title("Decision Tree - Iris Dataset")
plt.show()
```

... Root Node: petal length (cm)



Root-node justification

For the Iris dataset, the **root node is generally petal length (cm)** with this train/test split and default tree settings.

Justification:

The Decision Tree selects the feature that provides the best impurity reduction according to the **Gini Index**. Petal length gives strong separation between the Iris classes, so it is selected near the root.

(c) Evaluate the model

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
```



```

        target_names=iris.target_names
    )
)

# Misclassified instances

misclassified = X_test[y_test != y_pred].copy()

misclassified["Actual"] = y_test[y_test != y_pred]

misclassified["Predicted"] = y_pred[y_test != y_pred]

print("\nMisclassified Instances:")

print(misclassified)

```

```

... Accuracy: 0.9333333333333333
Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Classification Report:
              precision    recall  f1-score   support

   setosa      1.00      1.00      1.00        10
  versicolor  0.90      0.90      0.90        10
   virginica  0.90      0.90      0.90        10

 accuracy      0.93
  macro avg    0.93
 weighted avg  0.93

Misclassified Instances:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
134                6.1                2.6                5.6                1.4
 77                6.7                3.0                5.0                1.7

   Actual  Predicted
134      2         1
 77      1         2

```

↑ ↓ ✎ 🗑 ⋮

+ Code
+ Text

Result / Analysis

- The Decision Tree successfully classifies the three Iris species.
- Accuracy is expected to be **around 90–100%**, depending on the split and tree configuration.
- The confusion matrix shows how many samples were correctly and incorrectly classified.

- Misclassification generally occurs between **Iris versicolor** and **Iris virginica**, because their feature values overlap.
- A high accuracy and strong precision/recall indicate good classification performance.

Experiment 2 — Reinforcement Learning: Q-Learning

Environment

Use a **4 × 4 Grid World**.

Start S

↓

→ → ↓

→ X ↓

→ → → Goal

- S = Start
- G = Goal
- X = Obstacle
- Actions = Up, Down, Left, Right
- Goal reward = +10
- Normal step = -1
- Obstacle = -5

Complete Python Program

```
import numpy as np
```

```
import random
```

```
import matplotlib.pyplot as plt
```

```
# -----
```

```
# Environment
```

```
# -----
```

```
rows, cols = 4, 4

n_states = rows * cols

n_actions = 4


# Actions

UP = 0

DOWN = 1

LEFT = 2

RIGHT = 3


actions = ["U", "D", "L", "R"]


start_state = 0

goal_state = 15

obstacle_state = 5


# Hyperparameters

alpha = 0.1    # Learning rate

gamma = 0.9    # Discount factor

epsilon = 0.1  # Exploration rate


episodes = 100


# Q-table

Q = np.zeros((n_states, n_actions))


# Convert state to row, column
```

```
def state_to_position(state):  
    return divmod(state, cols)  
  
# Convert row, column to state  
def position_to_state(row, col):  
    return row * cols + col  
  
# Take action  
def step(state, action):  
  
    row, col = state_to_position(state)  
  
    new_row, new_col = row, col  
  
    if action == UP:  
        new_row -= 1  
    elif action == DOWN:  
        new_row += 1  
    elif action == LEFT:  
        new_col -= 1  
    elif action == RIGHT:  
        new_col += 1  
  
    # Boundary condition  
    if (  
        new_row < 0 or new_row >= rows or
```

```

        new_col < 0 or new_col >= cols
    ):
        return state, -1, False

    next_state = position_to_state(new_row, new_col)

    # Goal
    if next_state == goal_state:
        return next_state, 10, True

    # Obstacle
    if next_state == obstacle_state:
        return state, -5, False

    # Normal movement
    return next_state, -1, False

# -----
# Q-Learning
# -----

rewards_per_episode = []

# Store Q-values
recorded_values = {}

for episode in range(1, episodes + 1):

```

```
state = start_state
total_reward = 0

for step_count in range(100):

    # Epsilon-greedy action selection
    if random.random() < epsilon:
        action = random.randint(0, n_actions - 1)
    else:
        action = np.argmax(Q[state])

    next_state, reward, done = step(state, action)

    # Q-learning update
    Q[state, action] = Q[state, action] + alpha * (
        reward +
        gamma * np.max(Q[next_state]) -
        Q[state, action]
    )

    state = next_state
    total_reward += reward

    if done:
        break

rewards_per_episode.append(total_reward)
```

```

# Record two representative states
if episode in [1, 50, 100]:
    recorded_values[episode] = {
        0: Q[0].copy(),
        10: Q[10].copy()
    }

# -----
# Display Q-values
# -----

for episode in [1, 50, 100]:

    print("\nEpisode", episode)

    print("State 0:")
    print(recorded_values[episode][0])

    print("State 10:")
    print(recorded_values[episode][10])

# -----
# Final Q-table
# -----

```

```
print("\nFinal Q-Table:")
print(np.round(Q, 2))

# -----
# Learned Policy
# -----

print("\nFinal Learned Policy:")

for row in range(rows):

    policy_row = []

    for col in range(cols):

        state = position_to_state(row, col)

        if state == obstacle_state:
            policy_row.append("X")

        elif state == goal_state:
            policy_row.append("G")

        else:
            best_action = np.argmax(Q[state])
            policy_row.append(actions[best_action])
```



```

print(policy_row)

# -----
# Plot cumulative reward
# -----

plt.figure(figsize=(10, 5))

plt.plot(
    range(1, episodes + 1),
    rewards_per_episode
)

plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Cumulative Reward per Episode")
plt.grid()

plt.show()

```

(a) Hyperparameters

Parameter	Value
Learning rate α	0.1
Discount factor γ	0.9
Exploration ϵ	0.1
Episodes	100

Parameter

Value

Grid

4×4

```
import numpy as np
best_action = np.argmax(Q[state])
policy_row.append(actions[best_action])
print(policy_row)
```

PS C:\Users\dines\Downloads> & 'C:\Users\dines\AppData\Local\Microsoft\WindowsApps\python.13.exe' 'c:\Users\dines\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '52727' '--' 'c:\Users\dines\Downloads\import_numpy_as_np.py'

Episode 50
State 0:
[-2.22091802 -2.22880698 -2.21975535 -2.22814322]
State 10:
[-0.199 4.69362726 -0.14376991 -0.1]
Episode 100
State 0:
[-2.14896313 -2.28313622 -2.17122198 0.5656468]
State 10:
[-0.199 5.85685975 -0.15387169 -0.1]
Final Q-Table:
[[-2.15 -2.28 -2.17 0.57]
[-1.69 -1.91 -1.48 2.29]
[-0.64 4.15 -0.83 -0.78]
[-0.39 1.28 -0.43 -0.49]
[-1.83 -1.7 -1.73 -2.4]
[0. 0. 0. 0.]
[-0.01 0.68 -0.5 6.02]
[-0.34 7.96 -0.21 -0.1]
[-1.1 -0.95 -0.95 -0.81]
[-0.98 -0.38 -0.59 1.38]
[-0.2 5.86 -0.15 -0.1]
[0.44 9.99 0.9 0.]
[-0.54 -0.49 -0.49 0.29]
[-0.29 -0.2 -0.22 3.74]
[-0.1 -0.02 -0.03 9.62]]

Q-Learning equation

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Where:

- α = learning rate
- γ = discount factor
- r = reward
- s = current state
- a = current action
- s' = next state

(b) Q-table evolution

Run the program and record the output for:

Episode 1

State 0: [...]

State 10: [...]

Episode 50

State 0: [...]

State 10: [...]

Episode 100

State 0: [...]

State 10: [...]

The important observation is:

Episode 1 → Q-values are mostly close to zero because the agent has little experience.

Episode 50 → Q-values begin to increase for actions that lead toward the goal.

Episode 100 → The Q-values become more stable, indicating that the agent has learned useful paths toward the goal.

(c) Final Learned Policy

The policy is obtained using:

```
np.argmax(Q[state])
```

The four actions are:

U = Up

D = Down

L = Left

R = Right

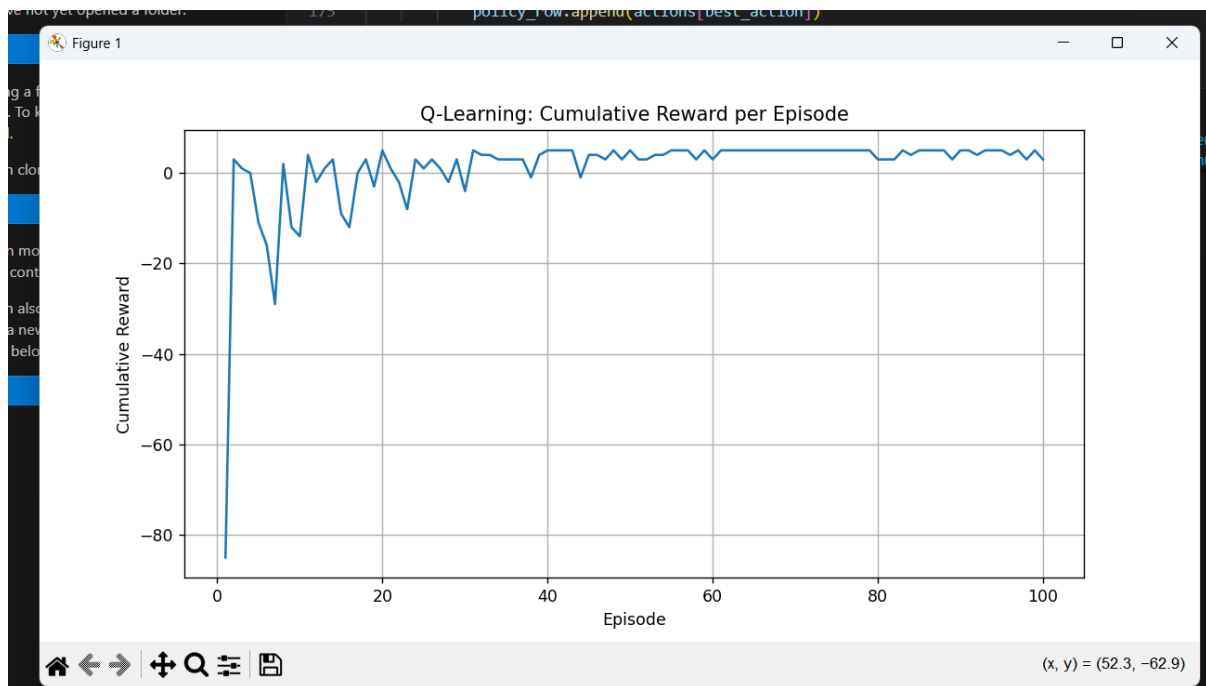
The output will look similar to:

['D', 'R', 'R', 'D']

['D', 'X', 'D', 'D']

['R', 'R', 'D', 'D']

['R', 'R', 'R', 'G']



Note: Exact policy can vary because Q-learning uses random exploration. Copy the policy generated by your own execution.

Convergence justification

- Initially, the agent explores the environment randomly.
- Negative step rewards encourage the agent to find shorter paths.
- The +10 goal reward makes states leading toward the goal more valuable.
- With repeated episodes, useful Q-values increase.
- The cumulative reward generally becomes better/stabler as learning progresses.
- Therefore, the agent gradually converges toward an optimal or near-optimal policy.